2025牛客寒假算法基础集训营4题解

目前包含**ABCDEFJIKL**

个人难度排序 $\mathrm{K} < \mathrm{I} < \mathrm{E} < \mathrm{B} < \mathrm{D} < \mathrm{C} < \mathrm{A} < \mathrm{L} < \mathrm{J} < \mathrm{F}$

BC题干一样放在一起讲,

简单题

K.Tokitsukaze and Shawarma

> Tokitsukaze and Shawarma
>
> 客人需要 x 个沙威玛，y 杯可乐，z 包薯条。
>
> 制作 1 个沙威玛需要 a 分钟；让饮料机倒可乐，每一杯可乐需要 b 分钟；让炸锅自动炸薯条，每一包薯条需要 c 分钟。这三件事可以同时进行。
>
> 请问客人至少需要等待几分钟。

题解

按题意输出 $x*a, y*b, z*c$ 中的最大值即可

```
t = int(input())
for _ in range(t):
    x, y, z, a, b, c = map(int, input().split())
    print(max(x*a,y*b,z*c))
```

I.Tokitsukaze and Pajama Party

> I-Tokitsukaze and Pajama Party
>
> 已知睡衣派对的第一句歌词为 `"u,u wa wa u wa"`
>
> 吉伊想问你，给你一个长度为 n、由小写字母组成的字符串 s，其中包含了多少个不同的 `"u*uwawauwa"` 歌词子串？其中字符 '`*`' 代表停顿，可以匹配任意个（至少一个）字母。**如果两个子串的位置不同，即视为不同。**
>
> 子串为从原字符串中，连续的选择一段字符（可以全选、可以不选）得到的新字符串。

题解

我们可以暴力遍历 s，寻找后半段子串 `"uwawauwa"` ，然后前方（不包括前一个字符）的 u 的数量即为改`"uwawauwa"` 贡献的歌词子串的数量。

某一位置之前的 u 的数量可以在预处理后快速得到。

```
for _ in range(int(input())):
    n = int(input())
    s = input()
    pre = [0 for i in range(n+4)]
    for i in range(n):
        pre[i+2] = pre[i+1] + (s[i] == 'u')
    ans = 0
    for i in range(n):
        if s[i:i+8] == 'uwawauwa':
            ans += pre[i]
    print(ans)
```

E.Tokitsukaze and Dragon's Breath

> E-Tokitsukaze and Dragon's Breath

有一张 n 行 m 列的网格地图，我们使用 (i,j) 表示网格中从上往下数第 i 行和从左往右数第 j 列的单元格。地图的每一个格子中都有一些怪物，我们使用 $a_{i,j}$ 表示坐标 (i,j) 处的怪物数量。

Tokitsukaze 坐在龙上，她可以让龙对着地图的任意一个坐标使出“龙之吐息”来打败怪物。“龙之吐息”会释放出火焰，火焰会打败坐标 (x,y) 内的所有怪物，并向 4 个方向：$(x-1,y-1)$, $(x-1,y+1)$, $(x+1,y-1)$, $(x+1,y+1)$ **一直延伸**，即火焰会在矩形中会形成一个 'X' 形状。火焰会打败经过的每个坐标中的所有怪物。

现在，Tokitsukaze 想知道，释放一次“龙之吐息”最多能打败多少只怪物。

题解

不难想到要预处理斜线上的和。

可以发现同一斜线上的元素其 $x+y$ 或 $y-x$ 是相同的，可以用 $x+y$ 或 $y-x$表示这一斜线。

每一点的答案即为两条斜线的和减去这一点的值。

遍历一遍取最大值即可。

```
t = int(input())
for _ in range(t):
    n, m = map(int, input().split())
    maze = [list(map(int, input().split())) for i in range(n)]
    tmaze = [maze[i][::-1] for i in range(n)] # 水平翻转后可套用同样的代码
    memoxy = {}
    memonxy = {}
    # y = x + k方向
    for k in range(m):
        s = 0
        for x in range(n):
            y = x + k
            if y >= m:
                break
            s += maze[x][y]
        memoxy[k] = s
    for k in range(1, n):
        s = 0
        for y in range(m):
            x = y + k
```

```
            if x >= n:
                break
            s += maze[x][y]
        memoxy[-k] = s
    # y = -x + k 方向
    for k in range(m):
        s = 0
        for x in range(n):
            y = x + k
            if y >= m:
                break
            s += tmaze[x][y]
        memonxy[k] = s
    for k in range(1, n):
        s = 0
        for y in range(m):
            x = y + k
            if x >= n:
                break
            s += tmaze[x][y]
        memonxy[-k] = s
    ans = 0
    for x in range(n):
        for y in range(m):
            ans = max(ans, memoxy[y-x] +memonxy[m-1-y-x] - maze[x][y])
    print(ans)
```

中等题

B/C.Tokitsukaze and Balance String (easy)/(hard)

> [B-Tokitsukaze and Balance String (easy)]

一个字符串是平衡的，当且仅当字符串中 "01" 连续子串的个数与 "10" 连续子串的个数相同。

定义字符串 s 的 $\mathrm{val(s)}$ 为这样的位置数量：

- 若当前位置上的字符为 '0'，将其反置后成为 '1'，此时字符串 s 是平衡的；
- 若当前位置上的字符为 '1'，将其反置后成为 '0'，此时字符串 s 是平衡的。

现在 Tokitsukaze 有一个长度为 n，仅由字符 '0'、'1'、'?' 构成的字符串 s。

其中，'?' 可以替换成 '0' 或者 '1'。假设 "?' 的个数为 p，显然将每个 '?' 都替换后，总共可以得到 2^p 个不同的 s，Tokitsukaze 想知道所有 2^p 个不同的 s 对应的 $\mathrm{val(s)}$ 之和是多少。由于答案可能很大，请将答案对 (10^9+7) 取模后输出。

子串为从原字符串中，连续的选择一段字符（可以全选、可以不选）得到的新字符串。

题解

easy版本中 $n \le 10$，因此 $O(n^2 * 2^n)$ 的纯暴力做发是可取的。

我们用 2 进制枚举的方法枚举每一种情况，再在每个位置翻转后判断是不是平衡的。（注意保留原状态）

```
def check(s):
```

```
    c = [0, 0]
    for i in range(len(s)-1):
        if s[i] == '1' and s[i+1] == '0':
            c[1] += 1
        if s[i] == '0' and s[i+1] == '1':
            c[0] += 1
    return 1 if c[0] == c[1] else 0

def cal(s):
    ans = 0
    for i in range(len(s)):
        s[i] = str(int(s[i])^1)
        ans += check(s)
        s[i] = str(int(s[i])^1)
    return ans

t = int(input())
for _ in range(t):
    n = int(input())
    s = input()
    num = s.count('?')
    s = list(s)
    if num == 0:
        print(cal(s))
    else:
        ans = 0
        for i in range(1<<num):
            ns = s[:]
            rep = str(bin(i))[2:]
            rep = '0' * (num - len(rep)) +rep
            p = 0
            for i in range(n):
                if ns[i] == '?':
                    ns[i] = rep[p]
                    p += 1
            t = cal(ns)
            ans += t
        print(ans%(10**9+7))
```

hard版本显然不能暴力。

对 s 来说，连续的 1 和连续的 0 显然不会对 "01" 和 "10" 的数量产生影响。

因此 s 可以等价为 "1010101..."

我们可以发现，中间的 0 和 1 对 "01" 和 "10" 的数量的贡献总是一样的。

因此有

- 首尾相等的 s 一定是平衡的
- 首尾不相等的 s 一定是不平衡的

1） 若 s 长度为 1

- $s = ?$, 答案为 2
- $s\,! = ?$, 答案为 1

2）若 s 长度大于 1

- s 首尾相等，或其中至少有一个？，则除了首尾外的 $n-2$ 位置翻转后一定是平衡的
- s 首尾不相等，或其中至少有一个？，则翻转首尾 2个位置后首尾相等，此时一定平衡

```
t = int(input())
for _ in range(t):
    n = int(input())
    s = input()
    if n == 1:
        print(1 if s !='?' else 2)
    else:
        t = s.count('?')
        ans = 0
        if s[0] == s[-1] or (s[0] == '?' or s[-1] == '?'):
            ans += (1<<t)*(n-2)
        if s[0] != s[-1] or (s[0] == '?' or s[-1] == '?'):
            ans += (1<<t)* 2
        if s[0] == '?' or s[-1] == '?': # 去除重复计算的情况
            ans = ans//2
        print(ans%(10**9+7))
```

D.Tokitsukaze and Concatenate Palindrome

> [D-Tokitsukaze and Concatenate Palindrome]

Tokitsukaze 有一个长度为 n 的字符串 a 和一个长度为 m 的字符串 b。两个字符串均仅包含小写字母。随后，依次进行以下操作：

- 第一步，Tokitsukaze 可以将 a 重新排列（也可以不进行重新排列），再将 b 重新排列（也可以不进行重新排列）。例如，假设 $a=$ "xyz"，对 a 重新排列后，a 可以是："xyz", "xzy", "yxz", "yzx", "zxy", "zyx" 这 6 种字符串中的一种；
- 第二步，Tokitsukaze 可以进行若干次替换（可以为 0 次）：每次选一个字符串，再从中选一个位置，然后把该位置上的字母替换成任意小写字母；
- 第三步，将 a 与 b 拼接起来形成一个新的字符串 s。拼接时须让 a 在 b 之前，也就是说 $s=a+b$。

现在，Tokitsukaze 想知道，在第二步操作时，至少需要替换几次，才能使 s 为回文串。

一个字符串被称作回文串，当且仅当这个字符串从左往右读和从右往左读是相同的。例如，"a", "aa", "aba", "caac" 均是回文串。

题解

不难想到，我们可以匹配 a, b 中都包含的字符。

接下来，长的字符串中还可以继续匹配相同的字符，但不能超过 $\lfloor \frac{|n-m|}{2} \rfloor$ 个。（不然没有地方排列）

剩下的 k 个字符，我们更改 $\lfloor \frac{k}{2} \rfloor$ 个字符一定能使他们匹配。

```
from collections import Counter
t = int(input())
for _ in range(t):
    n, m = map(int, input().split())
    a = input()
    b = input()
```

```
    if n < m:
        n, m = m, n
        a, b = b, a
    ma = Counter(a)
    mb = Counter(b)
    for k in ma:
        t = min(ma[k], mb[k])
        ma[k] -= t
        mb[k] -= t
    maxcan = (n - m)//2
    for k in ma:
        if ma[k] >= 2:
            t = min(maxcan, ma[k]//2)
            maxcan -= t
            ma[k] -= 2 *t
        if maxcan == 0:
            break
    ans = 0
    for k in ma:
        ans += ma[k]
    for k in mb:
        ans += mb[k]
    print(ans//2)
```

困难题

A.Tokitsukaze and Absolute Expectation

> A-Tokitsukaze and Absolute Expectation

Tokitsukaze 有一个由 n 个整数组成的序列 $\{a_1, a_2, \ldots, a_n\}$，其中第 i 个元素 a_i 的值在区间 $[l_i, r_i]$ 中独立地、等概率随机生成。

定义一个序列的价值为 $\mathrm{val}(a) = \sum\limits_{i=2}^{n} \left|a_{i-1} - a_i\right|$，Tokitsukaze 想知道 $\mathrm{val}(a)$ 的期望。

题解

易得 $E(val(a)) = \sum\limits_{i=2}^{n} E(\left|a_{i-1} - a_i\right|)$

又有 $E(\left|a_{i-1} - a_i\right|) = \sum\limits_{i=l1}^{r1} \sum\limits_{i=l2}^{r2} \left|i - j\right|$

令 $l_1 < l_2$，先考虑两者互不包含的情况

- 若 $l_2 < r_1$，求和可得此时的期望为 $\frac{(r2-l2+1)\cdot(r1-l1+1)\cdot(r2+l2-r1-l1)}{(r2-l2+1)\cdot(r1-l1+1)} = r2 + l2 - r1 - l1$
- 若 $l_2 >= r_1$，可拆为 $[l_1, l_2 - 1]$ 与 $[l_2, r_2]$ 、$[l_2, r_1]$ 与 $[l_2, r_1]$、$[l_2, r_1]$ 与 $[r_1 + 1, r_2]$，第一组和第三组

  计算方式和上一种相同，中间一组求和后为 $2 * \sum\limits_{i=1}^{r1-l2+1} \frac{i^2+i}{2} = (r1 - l2 + 1)^3 - (r1 - l2 + 1)$

期望为 $\frac{(r1-l2+1)^3-(r1-l2+1)}{(r2-l2+1)\cdot(r1-l1+1)}$

其余情况讨论后可发现同理

```
MOD = 10**9+7

def pow(x, p):
    res = 1
    x %= MOD
    while p:
        if p & 1:
            res = res * x %MOD
        x = x * x % MOD
        p >>= 1
    return res

inv3 = pow(3, MOD-2)
inv2 = pow(2, MOD-2)

def num1(l, r):
    return ((r-l+1)*(r-l+1)%MOD*(r-l+1)%MOD - (r-l+1) + MOD) % MOD * inv3 % MOD

def num2(l1, r1, l2, r2):
    if l1>r1 or l2>r2:
        return 0
    res = (r2-l2+1)*(r1-l1+1)*(r2+l2-r1-l1)%MOD *inv2%MOD
    return (res+MOD)%MOD

def cal(l1, r1, l2, r2):
    if l1 > l2:
        l1, l2 = l2 , l1
        r1, r2 = r2, r1
    x = max(l1, l2)
    y = min(r1, r2)
    if x > y:
        return num2(l1, r1, l2, r2)
    ans =  num1(x, y)
    ans += num2(l1, x-1, l2, r2)
    ans += num2(x, y, y + 1, max(r1, r2))
    return ans % MOD

def solve():
    n = int(input())
    a = [list(map(int, input().split())) for i in range(n)]
    ans = 0
    for i in range(n-1):
        fz = cal(*a[i], *a[i+1])%MOD
        fm =  pow((a[i][1]-a[i][0] + 1)*(a[i+1][1]-a[i+1][0] + 1)%MOD, MOD-2)
        ans = (ans + fz * fm %MOD)%MOD
    print(ans%MOD)

for _ in range(int(input())):
    solve()
```

L.Tokitsukaze and XOR-Triangle

> [L. Tokitsukaze and XOR-Triangle]
>
> Tokitsukaze 有两个由 n 个整数组成的序列 $\{a_1, a_2, \ldots, a_n\}$ 和 $\{b_1, b_2, \ldots, b_n\}$。
> 现在，他有 q 次询问，每次询问给出两个整数 l 和 r，他想要知道：

```
$\sum\limits_{i=l}^r \sum\limits_{j=i}^r a_i \oplus b_j$
```

题解

由于是异或和，不难想到按位求前缀和。

$\sum_{i=l}^{r}\sum_{j=i}^{r} a_i \oplus b_j$

= $\sum_{i=l}^{r}\sum_{j=1}^{r} a_i \oplus b_j - \sum_{i=l}^{r}\sum_{j=1}^{i-1} a_i \oplus b_j$

第一部分 $\sum_{i=l}^{r}\sum_{j=1}^{r} a_i \oplus b_j$ ，可以按照经典做法做（拆位后统计 0 和 1 的个数）。

第二部分$\sum_{i=l}^{r}\sum_{j=1}^{i-1} a_i \oplus b_j$ 可以预处理 $\sum_{i=1}^{r}\sum_{j=1}^{i-1} a_i \oplus b_j$ 后快速得出（代码中的prec)

```
#include <bits/stdc++.h>
using namespace std;

#define pb push_back
#define eb emplace_back

typedef long long ll;
typedef pair<long long,long long> PLL;
typedef tuple<ll,ll,ll> TLLL;
const ll inf =  0x3f3f3f3f;
const ll INF = INT_MAX;
const ll MOD = 1000000007;

inline ll read(){
    ll s = 0, w = 1; char ch = getchar();
    while (ch < 48 || ch > 57) { if (ch == '-') w = -1; ch = getchar(); }
    while (ch >= 48 && ch <= 57) s = (s << 1) + (s << 3) + (ch ^ 48), ch =
getchar();
    return s * w;
}

inline void pt(ll x){if(x<0) putchar('-'),x=-x;if(x>9)
pt(x/10);putchar(x%10+'0');}

inline void print(ll x){pt(x), puts("");}

inline void printVector(vector<ll> vec){
    for(const auto t:vec){
        pt(t),putchar(' ');
    }
    puts("");
}

ll pow2[55];
void solve(){
```

```
    ll n = read(), q = read();
    vector<ll> a(n+1),b(n+1);
    vector<vector<ll>> prea(n+1, vector<ll> (32, 0)) ,preb(n+1,vector<ll> (32,
0));
    vector<vector<ll>> prec(n+1, vector<ll> (32, 0));
    for(ll i=1;i<=n;i++) a[i] = read();
    for(ll i=1;i<=n;i++) b[i] = read();
    for(ll bit=31;bit>=0;bit--){
        for(ll i=1;i<=n;i++){
            prea[i][bit] = prea[i-1][bit]+((a[i]>>bit)&1);
            preb[i][bit] = preb[i-1][bit]+((b[i]>>bit)&1);
            prec[i][bit] = (prec[i-1][bit] + ((a[i]>>bit)&1) * (i-1-preb[i-1]
[bit]) + (((a[i]>>bit)&1)^1) * preb[i-1][bit])%MOD;
        }
    }
    while (q--){
        ll ans = 0;
        ll l = read(), r = read();
        for(ll bit=31;bit>=0;bit--){
            ll numa1 = prea[r][bit] - prea[l-1][bit];
            ll numb1 = preb[r][bit];
            ll t1 = numa1 * (r - numb1) + (r-l+1-numa1) * numb1;
            ll t2 = prec[r][bit] - prec[l-1][bit];
            ans = (ans + (t1 - t2 + MOD)%MOD*pow2[bit]%MOD)%MOD;
        }
        print(ans);
    }
}

int main(){
    pow2[0] = 1;
    for(ll i=1;i<=50;i++){
        pow2[i] = (pow2[i-1] * 2 )%MOD;
    }
    ll t = 1;
    t = read();
    while(t--){
        solve();
    }
}
```

F.Tokitsukaze and Kth Problem (easy)

> Tokitsukaze and Kth Problem (easy)

Tokitsukaze 有一个由 n 个整数组成的序列 $\{a_1, a_2, \ldots, a_n\}$。

他定义一个二元组 (i, j) 满足 $1 \le i < j \le n$，并定义这个二元组对应的值 $f(i, j) = (a_i + a_j) \bmod p$。

他想知道，对于全部不同的二元组 (i, j)，全部值 $f(i, j)$ 的前 k 大值分别为多少。如果不存在，则输出 -1 替代。

```
$2 \leq n \leq 2 \times 10^5;\ 1 \leq p \leq 10^9;\ 1 \leq k \leq 2 \times 10^5$
```

题解

由题意 $1 \leq k \leq 2 \times 10^5$ ，因此我们计算出 k 个最大值是可行的。

使用最大堆，每次取出当前的寻找下一个最优解。

由于题目的答案是对 p 取模的，因此我们可以对每个 a_i 对 p 取模。

先对 a 升序排序，则 a_i 与右边的元素的 $f(i,j)$ 的值会以 $p-a_i$ 左边的元素(设为 pos)为分界线，且两侧的随着 j 的减少 $f(i,j)$ 均在减少。

于是我们开始把两个值 $j=pos, j=n$ 都放入堆中，每次左移在插入即可。

```
#include <bits/stdc++.h>
using namespace std;
#define pb push_back
typedef long long ll;
typedef pair<long long,long long> PLL;
inline ll read(){
    ll s = 0, w = 1; char ch = getchar();
    while (ch < 48 || ch > 57) { if (ch == '-') w = -1; ch = getchar(); }
    while (ch >= 48 && ch <= 57) s = (s << 1) + (s << 3) + (ch ^ 48), ch =
getchar();
    return s * w;
}
inline void pt(ll x){if(x<0) putchar('-'),x=-x;if(x>9)
pt(x/10);putchar(x%10+'0');}
inline void print(ll x){pt(x), puts("");}
inline void printVector(vector<ll> vec){
    for(const auto t:vec){
        pt(t),putchar(' ');
    }
    puts("");
}

void solve(){
    ll n = read(), p = read(), k = read();
    vector<ll> a(n+1, 0);
    for(ll i=1;i<=n;i++) a[i] = read()%p;
    sort(a.begin()+1, a.end());
    priority_queue<PLL> pq;
    vector<ll> vis1(n+1, 0), vis2end(n+1, 0), vis2(n+1, n);
    for(ll i=1;i<=n;i++){
        ll pos = lower_bound(a.begin()+i, a.end(), p-a[i]) - a.begin() - 1;//p -
a[i] 的 前一个
        vis1[i] = pos;
        vis2end[i] = max(pos, i);// 遍历过程不重复遍历
        if (pos > i){
             pq.push({(a[i] + a[pos])%p, i});
        }
        pq.push({(a[i] + a[n])%p, i});
    }
    vector<ll> ans;
    while((ll)ans.size()<k){
```

```
        if (pq.empty()){
            ans.pb(-1);
            continue;
        }
        auto [val, idx] = pq.top();
        pq.pop();
        if (vis1[idx] > idx&&val == (a[idx] + a[vis1[idx]])%p){
            ans.pb(val);
            vis1[idx] --;
            if (vis1[idx] > idx){
                pq.push({(a[idx] + a[vis1[idx]])%p, idx});
            }
        }
        else{
            if(vis2[idx] > vis2end[idx]){
                ans.pb(val);
                vis2[idx]--;
                if(vis2[idx] > vis2end[idx]){
                    pq.push({(a[idx] + a[vis2[idx]])%p, idx});
                }
            }
        }
    }
    printVector(ans);
}

int main(){
    ll t = 1;
    t = read();
    while(t--){
        solve();
    }
}
```

J.Tokitsukaze and Recall

> [Tokitsukaze and Recall]

Tokitsukaze 正在玩“岛屿”地图，地图上的区域并不一定连通。现在敌方部队占领着 n 块区域，并有 m 条双向道路，第 i 条连接着两片不同的区域。

Tokitsukaze 想要占领敌方区域，需要派遣部队清理该区域内所有的敌方单位。我方部队可以在我方已占领的区域内随意移动，但无法穿过敌方占领的区域。也就是说，假设当前玩家的部队在区域 S，并想让部队到达区域 E，那么需要至少存在一条从区域 S 到区域 E 的路径，这条路径上所有区域均是我方已占领的区域。但还有另一种方法，就是使用仲裁者的技能。在区域 E 部署一个仲裁者，这样玩家可以通过区域 E 部署的仲裁者的技能 "recall"，直接把在任意区域上的部队直接传送到 E。

由于黄金舰队的造价太高，Tokitsukaze 打造了一个无敌的地面部队，她想使用这个部队尽可能多的占领敌方区域。此时部队在区域 0，该区域不与任何敌方区域相连接，所以需要在敌方区域部署至少 1 个仲裁者才能让部队到达某个敌方区域。但她打造地面部队时已经消耗了大量资源，只能使用剩余的资源建造出 k 个仲裁者。

现在 Tokitsukaze 打算合理部署这 k 个仲裁者，使她可以尽可能多的占领敌方区域。Tokitsukaze 想知道她最多能占领多少敌方区域，并且按占领的顺序输出区域编号，如果有多种方案，输出字典序最小的那种。

关于字典序：数组 c 在字典序上小于数组 d 当且仅当以下任一情况成立：

- $|c| < |d|$ 并且对于所有 $1 \le i \le |c|$ 满足 $c_i = d_i$，其中 $|c|$ 表示数组 c 的长度；
- 在 c 和 d 不同的第一个位置上，数组 c 的元素小于数组 d 中对应的元素。

题解

我们可以用并查集获取所有的连通分量，为了最大化数量，不难想到从点数多的并查集开始取。

我们可以用BFS来获取顺序，同时为了最小化字典序，如果还有技能次数，我们应当直接跳到前一次的下一个。

并且，BFS的队列要改成有序的，每次取出最小值。

```
#include <bits/stdc++.h>
using namespace std;

#define pb push_back
#define eb emplace_back

typedef long long ll;
typedef pair<long long,long long> PLL;
typedef tuple<ll,ll,ll> TLLL;
const ll inf =  0x3f3f3f3f;
const ll INF = INT_MAX;
const ll MOD = 1000000007;
const ll mod = 998244353;

inline ll read(){
    ll s = 0, w = 1; char ch = getchar();
    while (ch < 48 || ch > 57) { if (ch == '-') w = -1; ch = getchar(); }
    while (ch >= 48 && ch <= 57) s = (s << 1) + (s << 3) + (ch ^ 48), ch =
getchar();
    return s * w;
}

inline void pt(ll x){if(x<0) putchar('-'),x=-x;if(x>9)
pt(x/10);putchar(x%10+'0');}

inline void print(ll x){pt(x), puts("");}

inline void printVector(vector<ll> &vec){
    for(const auto t:vec){
        pt(t),putchar(' ');
    }
    puts("");
}

void solve(){
    ll n = read(), m = read(), k = read();
    vector<ll> parent(n+1, 0);
    iota(parent.begin(), parent.end(), 0);
    vector<ll> cnt(n+1, 1);
    function<ll(ll)> find = [&](ll x){
        if (x == parent[x]) return x;
        return parent[x] = find(parent[x]);
    };
```

```
    function<void(ll, ll)> Union = [&](ll x, ll y){
        ll fx = find(x);
        ll fy = find(y);
        if (fx == fy) return;
        if (fx > fy) swap(fx, fy);
        parent[fy] = fx;
        cnt[fx] += cnt[fy];
    };
    vector<vector<ll>> adj(n+1);
    for(ll i=1;i<=m;i++){
        ll u = read(), v = read();
        adj[u].eb(v);
        adj[v].eb(u);
        Union(u, v);
    }
    set<PLL> st;
    for(ll i=1;i<=n;i++){
        if (parent[i] == i){
            st.insert({cnt[i], -i});
        }
    }
    set<ll> pq;
    ll ans = 0;
    while(st.size()&&k){
        auto [val, idx] = *st.rbegin();
        st.erase(*st.rbegin());
        ans += val;
        pq.insert(-idx);
        k--;
    }
    ll now = 0;
    vector<ll> vis(n+1, 0);
    vector<ll> path;
    while(!pq.empty()){
        ll u = *pq.begin();
        if ((u!=(now+1))&&k){
            k--;
            now++;
            pq.insert(now);
            u = *pq.begin();
        }
        pq.erase(*pq.begin());
        vis[u] = 1;
        now = u;
        path.pb(u);
        for(auto v:adj[u]){
            if(!vis[v]){
                pq.insert(v);
            }
        }
    }
    print(ans);
    printVector(path);
}

int main(){
```

```
    ll t = 1;
    t = read();
    while(t--){
        solve();
    }
}
```